

APUNTESWEB · INGENIERÍA INFORMÁTICA

POO

Segundo curso · 1.º cuatrimestre · 8 temas

JULIO DE 2026 · [HTTPS://ADRIANQL5.GITHUB.IO/APUNTESWEB/](https://adrianql5.github.io/apuntesweb/)

1. Encapsulación

Escrito por **Adrián Quiroga Linares**. [adriangl5](#)

1.1 Tipos de datos

Los **tipos de datos** disponibles en un lenguaje de programación limitan enormemente la posibilidades a la hora de crear programas si no es posible representar un tipo de datos concreto, no se podrá operar sobre ellos. El **problema de C** se produce cuando intentamos definir nuevos tipos de datos, su **integridad es limitada**, no podemos **restringir el valor que toman las variables en distintas funciones del programa**.

1.1.1 Encapsulación

La **encapsulación** es una de las principales características de la **programación orientada a objetos**, puesto que asegura la **integridad** de los datos, limitando las porciones de código del programa que pueden acceder a determinados datos.

Hace uso del **principio de ocultación**, dónde sólo un trozo de código puede ver un conjunto de datos dado.

1.2 Clases

En **POO** los tipos de datos se conceptualizan como **clases** y sus valores concretos se denominan **objetos**. Por ejemplo *Asia* sería un objeto que pertenece a la clase *Continente*.

1.2.1 Clases en Java

En Java tenemos estos tipos de datos:

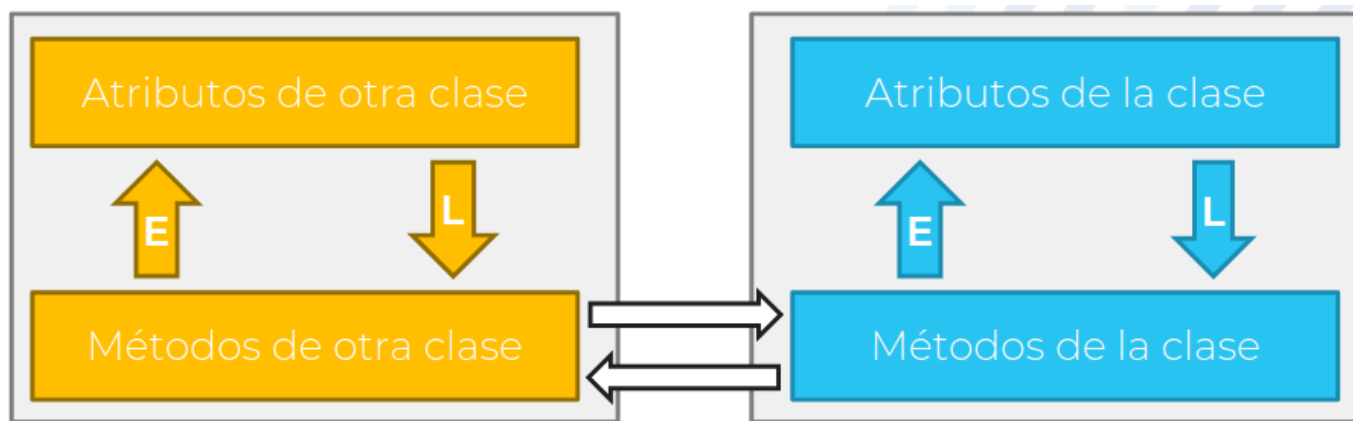
- **Tipos de datos primitivos:** vinculados a la representación de los datos en el ordenador, como los de `C`.
- **Clases:** vinculadas a las entidades de alto nivel del programa y no se pueden representar de forma directa en el ordenador.

1.2.2 Encapsulación en las Clases

Las clases contienen dos tipos de código:

- **Variables o Atributos:** que son las características que definen a la entidad representada por cada clase
- **Funciones o Métodos:** que acceden a los atributos para permitir su lectura y escritura, y para proporcionar la funcionalidad requerida en el programa.

Para resolver **el problema de integridad** de los datos, las **únicas funciones** que pueden acceder a los atributos, tanto para leerlos como para escribir sobre ellos, son las funciones de la clase.



Tenemos **cuatro tipos de acceso**, que se pueden tanto aplicar a los **atributos como a métodos**.

- **Público (public):** cualquier método de cualquier clase del programa puede acceder a los atributos y métodos.
- **Privado (private):** sólo métodos de la **misma clase** pueden acceder a los atributos
- **Acceso a paquete (no tiene especificador):** pueden acceder métodos de las clases que pertenezcan al **mismo paquete** (carpeta).
- **Acceso protegido (protected):** pueden acceder métodos de todas las clases que pertenezcan al **mismo paquete** y también **subclases** de la clase (aunque no estén en el mismo paquete)..

[!Buenas Prácticas]

- Los **atributos** de una clase deben ser siempre **privados**, para que no puedan ser modificados de forma incorrecta, y los **métodos deben ser públicos**, para poder acceder a los atributos de las clases.
- Se pueden usar **métodos privados** para facilitar la codificación de otros métodos.

1.2.3 Estructura de una Clase

Getters y Setters

Los **getters** son métodos que **devuelven el valor** que tienen los atributos en cada momento de la invocación:

- Sólo hay un **único** getter para cada atributo:
- `get nombreAtributo`
- Por normal general contienen sólo código asociado a la devolución del atributo

Los **setters** son métodos que **escriben el valor** que tienen los atributos:

- Sólo hay un **único** setter para cada atributo
- `set nombreAtributo`
- Normalmente contienen código sobre **las condiciones que debe cumplir el argumento** para que sea un valor correcto del atributo.

[! Buenas Prácticas]

- Los **setters** siempre deben incluir una condición para evitar la escritura de **null** como valor de los atributos.
- Normalmente, **todos los atributos deben tener getters y setters**, salvo que solamente sean usados como parte de programación de los métodos de la clase.

Constructores

Al declarar un **atributo** de una clase, si este es un **tipo de dato primitivo**, se reservará memoria para él y se le asignará un valor inicial. Sin embargo, si es una **clase**, **no se reservará memoria** (*ya que se desconoce cuánto ocupará*) y se le asignará el valor inicial `null`.

Si se intentase usar el atributo en este estado, se generaría un error de ejecución

`NullPointerException`. Se debe reservar memoria para él invocando a los constructores con el operador `new`.

Al declarar una instancia de una **clase**, no se reservará memoria para dicho objeto tampoco. Si no se inicializa con el operador `new`, provocará un error de compilación.

Un **constructor** es un método que **no devuelve ningún tipo de dato**. Se invoca para:

- **Reservar memoria** para un objeto y sus atributos.
- **Asignar valores** iniciales a sus atributos

Su **nombre** coincide con el nombre de la clase a la que pertenece.

Se **invoca** una **única vez** para cada objeto porque cuando se reserva memoria para un objeto, su nombre es una referencia a la posición de memoria donde se almacenan los datos. Si se volviese a invocar al constructor, la referencia apuntaría a una nueva posición y se perdería la anterior.

Constructor por defecto:

- Java proporciona automáticamente este constructor cuando no se especifica ningún otro.
- No tiene **ningún argumento**

- Inicializa los atributos a sus **valores por defecto**, es decir, todos los atributos de tipo clase tomarán el valor `null`

[!Buenas Prácticas]

- Se debe evitar el constructor por defecto
- Los constructores creados deben tener como **argumentos** los valores de los **atributos** que es **obligatorio** inicializar para crear un objeto.
- Se pueden meter comprobaciones en el constructor

Métodos Funcionales

Los **métodos funcionales** son aquellos que utilizan los atributos de una clase para poder realizar las operaciones que implementan la funcionalidad de esta.

- `objeto.método()` es la forma de invocarlos

Métodos sobrecargados

- Decimos que un método funcional (o un constructor) de una clase está **sobrecargado** si tiene varias implementaciones.
- Todas las implementaciones **deben tener el mismo nombre**
- Todas las implementaciones **deben tener el mismo objetivo**
- Los **argumentos** deben ser **distintos** (si no, el compilador no podría distinguir cuál se está invocado y daría un fallo de compilación)

Método toString

- `toString` es un método que devuelve la **representación en texto** de un objeto de una clase
- Las clases heredan este método de la **clase Object**, la superior en la jerarquía de clases. Esta implementación es muy **genérica** y no aporta información sobre el **contenido del objeto**.
- Por ello, se recomienda **reimplementarlo** usando el operador `@Override` de manera que devuelva una representación en texto que resuma adecuadamente el contenido del objeto

[!Buenas Prácticas] Se **desaconseja** el uso de **condiciones** sobre los argumentos de un método para **seleccionar qué código** se debe ejecutar en cada caso. Es mejor crear distintas implementaciones del método.

1. 3 Registros

Un **registro** es una clase cuyas instancias son **inmutables**, es decir, los valores de los atributos no pueden cambiar en tiempo de ejecución. Todos sus atributos son **privados** y contienen su

correspondiente **getter**. **No tienen setters** y no puede existir ningún método que modifique sus valores.

Se pueden introducir nuevos **métodos** siempre que no alteren los valores de los atributos. Además tienen un **constructor canónico** con sus correspondiente argumentos para cada uno de los atributos.

- Se puede **reimplementar** especificando todos los argumentos o usando una **declaración compacta**.
- Se puede introducir **sobrecargas** siempre que en su cuerpo se invoque al constructor canónico del registro

El método `toString` incluye el **nombre de la clase** y el **nombre de los atributos** con sus correspondientes **valores**.

```
public record Continente(ArrayList<Pais> paises,
                        String nombre,
                        String color) {

    // Reimplementar constructor canónico: forma compacta
    public Continente {
        Objects.requireNonNull(paises);
        Objects.requireNonNull(nombre);
        Objects.requireNonNull(color);
    }

    // Nuevo constructor (debe invocar al canónico)
    public Continente(ArrayList<Pais> paises, String nombre) {
        this(paises, nombre, "Blanco");
    }

    // Método funcional: no puede escribir sobre los atributos
    public boolean esGrande() {
        if (paises.size() > 10) return true;
        return false;
    }
}
```

NOTA

No confundir con `public final class Contienente{};` esto solamente sirve para que no se pueda extender la clase y ya se verá más adelante.

1. 4 Ejecución de un programa en Java

Todo programa en Java tiene una **clase principal** en la que se encuentra el método **main**, que será el punto a partir del cual se inicia la ejecución del programa.

```
public static void main (String[] args)
```

La clase principal no debe tener ningún atributo.

```
public class RiskETSE{  
    public static void main(String[] args){  
        new Menu();  
    }  
}
```

La clase *Menú* analiza los comandos que introduce el usuario y genera los objetos que invocarán los métodos con los que se da respuesta a las funcionalidad del programa

2. Clases y tipos de datos

Escrito por **Adrián Quiroga Linares**.

2.1 Tipos de Datos Primitivos

- Tienen un tamaño **fijo**
- Tienen una **correspondencia directa** con los tipos de datos que es posible representar en un computador
- En el momento de su declaración se realiza automáticamente la **reserva de memoria** para la variable
- **NO TIENEN MÉTODOS**

2.1.1 Wrappers

Los tipos de datos primitivos **no son clases**, puesto que no encapsulan ni datos ni métodos que acceden y modifican dichos datos, sino que están directamente vinculados a los **valores de las variables**.

Para hacer consistente el esquema de tipos de datos de Java con programación orientada o objetos de cada tipo de datos primitivos se han definido **wrappers**, que proporcionan **métodos** que facilitan operaciones comunes sobre los datos

Los métodos de los wrappers de los tipos primitivos permiten **obtener** el valor del dato; **convertir** el dato a una cadena de texto y viceversa; o **convertir** el dato a otros tipos de datos primitivos.

```
Integer a = new Integer(10);
System.out.println("Valor de a en decimales: " + a.doubleValue());
Integer b = new Integer("18");
System.out.println("Valor de b: " + b);
```

Esto nos daría como salida 10.0 y 18.

Cosas que una vez sepas lo que es el aliasing te serán útiles

El aliasing de los wrappers

Aliasing ocurre cuando dos o más referencias apuntan al mismo objeto en memoria. En Java, los wrappers (como `Integer`, `Double`, `Character`, etc.) son objetos, por lo tanto es posible que dos variables apunten al mismo objeto wrapper.


```
Integer a = new Integer(5);
Integer b = a; // b y a apuntan al mismo objeto
b = 10;        // Ahora b apunta a un nuevo objeto, a sigue apuntando al original
```

En este ejemplo, al principio, **a** y **b** apuntan al mismo objeto (`Integer(5)`), por lo que hay aliasing.

Nota:

- Los wrappers en Java son **inmutables**, es decir, no puedes cambiar el valor interno después de la creación.
- Sin embargo, puedes cambiar a qué objeto apunta la referencia, pero no el contenido del objeto en sí.
- El aliasing ocurre entre referencias, no entre los valores internos.

```
Integer x = Integer.valueOf(100);
Integer y = x;        // Alias: x y y apuntan al mismo objeto Integer
System.out.println(x == y); // true

y = Integer.valueOf(200); // Ahora y apunta a otro objeto
System.out.println(x == y); // false
```

- **Sí, puede haber aliasing entre wrappers en Java.**
- Pero, dado que los wrappers son inmutables, el valor interno no puede cambiar a través del alias, solo la referencia puede cambiar de objeto.

La macabra integer pool

En Java, existe una especie de **"pool" de objetos Integer** para valores en el rango de **-128 a 127** (inclusive). Este mecanismo es similar al **String pool** pero aplicado a ciertos wrappers numéricos como `Integer`, `Short`, `Byte`, `Character` y `Long`.

Cuando se crea un `Integer` **Java reutiliza objetos existentes para valores entre -128 y 127**. Si usamos el operador `==` para comparar dos `Integer` dentro de ese rango, dará `true` si ambos provienen del pool. `==` **COMPARA LAS REFERENCIAS NO EL VALOR**.

```
Integer a = 100;
Integer b = 100;
System.out.println(a == b); // true (ambos usan el pool)

Integer c = 128;
Integer d = 128;
System.out.println(c == d); // false (no usan el pool, son objetos distintos)
```

El **Integer pool** mejora el rendimiento y reduce el uso de memoria para valores pequeños que son muy usados.

Respuesta corta:

Sí, en el rango de -128 a 127 los `Integer` se comportan como el "String pool": se reutilizan los objetos para ahorrar memoria y acelerar comparaciones.

Autoboxing & Unboxing

El uso de wrappers proporciona a Java gran **versatilidad** en el manejo de datos, pero genera código **difícil de entender**. El uso de **autoboxing y unboxing** resuelve este problema, ya que permite tratar un wrapper como si fuese un tipo de dato primitivo, y viceversa; pudiendo elegir en cada momento el comportamiento que se desee.

- **Autoboxing:** convierte un **tipo primitivo** a un objeto de la correspondiente clase **wrapper**. Lo usamos cuando:
 - Un método tiene como argumento un wrapper recibe un valor de **tipo primitivo**
 - A un **tipo primitivo** se le asigna un objeto de un wrapper
- **Unboxing:** convierte un objeto de una clase **wrapper** al **tipo primitivo** correspondiente. Se usa cuando:
 - Un método que tiene como argumento un **tipo primitivo** recibe un objeto un **wrapper**.
 - A un **tipo primitivo** se le asigna un objeto de un **wrapper**.

```
Integer a = 10; //autoboxing
int a = new Integer(10)//unboxing
```

[!Buenas Prácticas]

- Normalmente, es **preferible** usar **tipos primitivos** porque simplifican el código
- Es mejor usar **wrappers** para **convertir entre tipos primitivos**, sobre todo cuando en esa conversión se manejan **cadenas de texto**.

2.2 Referencias

En Java los **nombres de los objetos son referencias** a la posición de memoria que está reservada para ellos.

- Cuando se asigna un **objeto** `A` otro **objeto** `B`, en realidad **no se realiza una copia** de la memoria que ocupa `A` en la posición de memoria que ocupa `B`.
- Se está realizando una **asignación de la referencia del objeto** `A` **a la referencia al objeto** `B`, lo que es decir, ambas referencias apuntan a la misma posición de memoria.

- La memoria del objeto `A` ya **no está disponible**

2.2.1 Aliasing

- El **aliasing** es la asignación de referencias entre dos objetos.
- Es una de las principales características de la mayoría de los lenguajes orientados a objetos
- **Evita la encapsulación** de datos pues permite modificar el valor de atributos desde métodos de fuera de sus clases. Por tanto, los programas son mucho más difíciles de mantener, ya que los atributos de tipo objeto se pueden modificar sin ningún tipo de control de su clase

Otro ejemplo:

```
public static void main (String[] args){
    Jugador jugador = new Jugador("Luis", Valor.EJERCITO_AZUL, mapa);
    ArrayList <Pais> paises = jugador.getPaises(); //aliasing
    paises.remove(0)
}
```

Cómo y cuándo evitar el aliasing

Es muy **difícil desarrollar** programas en los que nunca se use el aliasing, además **evitarlo reduce el rendimiento**, por lo que se debe seleccionar muy bien cuándo se evitará, pues no siempre es adecuado hacerlo.

El aliasing sólo se puede evitar si se introduce manualmente código que genere una **nueva referencia del mismo objeto**. Este código tiene que:

- **Reservar** memoria para el objeto
- **Copiar** los **atributos** del objeto (si son objetos a su vez, se debe generar una nueva referencia para ellos también)

Método clone: El método `clone` genera una **copia exacta** de un objeto y la almacena en una **posición diferente** de la que ocupa el objeto original. El programador debe implementar el método para cada clase de cuyos objetos se desea realizar copias profundas.

- **Copia profunda:** se reserva memoria para todos los atributos del objeto, incluso todo los **elementos de un conjunto de datos**

```
@Override
public Object clone(){
    try{
        super.clone();
    }
}
```

```

        catch(CloneNotSupportedException exc){
            System.out.println(exc.getMessage());
        }
        Jugador jugador = new Jugador(nombre, color, (Mapa) mapa.clone());
        ArrayList<Pais> paisesClonados = new ArrayList<>();

        for(int i; i<paises.size();i++){
            paisesClonados.add((Pais) paises.get(i).clone)
        }

        return jugador
    }
}

```

2.3 Máquina Virtual de Java

Java es un lenguaje interpretado cuya ejecución corre a cargo de lo que se denomina **máquina virtual de Java**.

2.4 Almacenamiento

2.4.1 Almacenamiento de datos

Dependiendo del **tipo de dato** y del **lugar** del programa en el que se definen, los datos se almacenan en zonas de memoria diferentes.

Pila

Almacena datos relativos a la **ejecución de un método**, cuando este finaliza se **eliminan automáticamente** de memoria. Los datos que se almacenan en la pila siempre deben tener **tamaño conocido** (*el compilador necesita conocer con antelación cuánta memoria se va a reservar en la pila para mover el stack pointer al tope*):

- El **código** correspondiente a los métodos (*call stack*)
- Todos los **tipos primitivos** usados durante la ejecución de los métodos
 - Variables **locales**
 - Valores de los **argumentos**
 - Valores de **retorno**
 - Resultados **parciales**
- Las **referencias a objetos en el programa**

Montón (Heap)

Zona de la memoria en la que el procesador no necesita conocer qué **cantidad de datos** se va a reservar ni cuánto **tiempo** van a estar disponibles

- **Objetos creados a lo largo de la ejecución del programa** -> incluyendo sus atributos aunque sean **primitivos** (incluyen a los wrappers, strings y arrays)

2.4.2 Almacenamiento de Métodos

Una clase no ocupa memoria. Las instrucciones de los métodos se cargan en memoria (en la pila) cuando se **crea un objeto** de la clase a la que pertenecen.

2.5 Recolector de Basura

Se encarga de buscar en la memoria del programa para **identificar qué objetos se encuentra en uso y cuales no, y eliminar los objetos que no se usen mas.**

- Corre en segundo plano tras el lanzamiento del programa
- **Impacto directo en el rendimiento de un programa** porque la eliminación de los que no se usan facilita el acceso a objetos que se están utilizando.
- Realiza una gestión **más eficiente de la memoria** (*evitandonos tener que hacer frees como en C*)

2.5.1 Proceso Básico de Marcado y Borrado

1. **Marcado:** Se identifican qué zonas de la memoria están siendo usadas y cuáles no, lo cuál puede ser **muy ineficiente** si se deben analizar todos los objetos en el sistema
2. **Borrado normal (op1):** se eliminan de memoria los objetos que no tienen referencias asignadas durante más tiempo y mantiene una **lista** de posibles referencias a la parte de la **memoria** que puede ser utilizada.
 - **Borrado con compactación (op1):** para mejorar rendimiento, además de borrar los objeto no referenciados, se compacta la memoria, moviendo los objetos referenciados a posiciones de memoria consecutivas.

Sin embargo las operaciones de marcado y compactación **son muy ineficientes**, por lo que debemos cambiar el esquema de gestión de memoria.

2.5.2 Estructura

En la mayoría de los programas, el uso y la eliminación de los objetos **no sigue un comportamiento uniforme** a lo largo del tiempo:

- El tiempo de supervivencia de los objetos es pequeño
- A medida que pasa el tiempo, cada vez se mantienen en memoria menos objetos. Dividimos la memoria en varias partes, llamadas **generaciones**, para facilitar la gestión de la vida de los objetos.
- **Young generation:** donde se almacenan los objetos recién creados, a los que se les asigna una fecha. Está formada por el **eden space** y el **survivor space** (S_0 y S_1).
 - Cuando se llena se lanza una recolección de basura menor donde todos los threads se paran (**RB MINOR**). Es muy rápida y puede ser optimizada si se tienen que eliminar muchos objetos.
 - Los objetos que hayan sobrevivido hasta llegar a la **edad umbral** se trasladan a la generación vieja.
- **Old generation:** almacena los objetos de larga duración
 - De manera mucho menos frecuente se lanza una recolección de basura mayor donde todos los threads se paran (**RB MAYOR**). Es mucho más lenta que la menor porque involucra a todos los objetos vivos.
- **Permanent generation:** almacena las clases y métodos usados durante la ejecución del programa
 - Se llena en tiempo de ejecución con metadatos sobre las clases que se utilizan durante la ejecución del programa (**carga dinámica**)
 - En versiones posteriores se sustituye por el **metaspace**

2.5.3 Funcionamiento del método Avanzado

1. Al inicio del programa la **generación joven** está vacía. Cualquier **objeto recién** creado siempre se almacena inicialmente en el **eden**.
2. Cuando el **eden se llena**, se lanza una **recolección de basura menor** en la que se eliminan objetos que no se van a usar (*con referencia null*).
3. En la **RB minor**, todos los objetos que son usados (*referenciados*) se mueven al espacio S_0 , mientras que los objetos no usados (*no referenciados*) se eliminan del espacio **eden**.
4. En la siguiente **RB minor**, los objetos usados del **eden** se mueven al espacio S_1 , y los objetos del espacio S_0 se mueven al espacio S_1 aumentando su edad, y se eliminan del **espacio** S_0 los no usados.
5. En la siguiente **RB minor**, todos los objetos usados del **eden** se mueven al espacio S_0 , mientras que los usados del espacio S_1 se mueven al espacio S_0 aumentando su edad, y se eliminan del espacio S_1 los objetos no usados.

6. En cada **RB minor** se comprueba si los objetos usados superan una determinada edad (8 en las imágenes) en cuyo caso **son promocionados** a la generación antigua, aumentando su edad

2.5.4 Gestión Avanzada de Referencias

Eliminar una referencia no elimina al objeto de la memoria, simplemente hace que deje de apuntar al objeto. Esto deja al objeto sin dueño, y por tanto puede ser eliminado por el recolector de basura.

```
MiObjeto obj = new MiObjeto();
obj = null; // Aquí, la referencia 'obj' deja de apuntar al objeto

MiObjeto obj = new MiObjeto();
obj = new MiObjeto(); // Ahora 'obj' apunta a un nuevo objeto, la referencia anterior se pierde

void miMetodo() {
    MiObjeto local = new MiObjeto();
    // ... aquí 'local' existe
} // Al salir del método, 'local' desaparece automáticamente

List<MiObjeto> lista = new ArrayList<>();
lista.add(new MiObjeto());
lista.remove(0); // Se pierde la referencia desde la lista
```

Existen **4 tipos de referencias** que se diferencian entre sí en **cómo las maneja el recolector de basura**. Son heredados de la clase `Reference`

- **Referencias fuertes:**

- Son el tipo de referencia por **defecto** en Java. Se crean automáticamente cuando se instancia un objeto, es decir, cuando haces algo como `Object obj = new Object()`.
- El recolector de basura **elimina** el objeto cuando no hay **ninguna referencia fuerte** apuntando a él (apunta a `null`).

- **Referencias débiles :**

- Se **crean** usando la clase `WeakReference` poniendo como **argumento** la referencia fuerte a la que apunta (al objeto al que apunta)
- Aunque se haya creado una referencia débil, el recolector de basura puede eliminar aun así la referencia fuerte a la que apunta

- Si sólo quedan referencias débiles a un objeto, el recolector de basura puede eliminar ese objeto.
- El **acceso** a la referencia fuerte no está asegurado, podría devolver `null`: se realiza con `get`
- El recolector de basura las **elimina** de memoria cuando no apuntan a una referencia fuerte o su referencia fuerte es `null`

3. Referencias suaves:

- Se crean usando la clase `SoftReference` poniendo como **argumento** la referencia fuerte a la que apunta (al objeto al que apunta).
- El **acceso** a la referencia fuerte está asegurado, aunque se elimine
- El recolector de basura elimina los objetos referenciados únicamente por referencias suaves sólo cuando el sistema necesita liberar memoria.

3. Referencias fantasmas:

- Se crean usando la clase `PhantomReference`, pasando el objeto y una `ReferenceQueue`
- El método `get()` siempre devuelve `null`.
- Sirven para ser notificados cuando el objeto está a punto de ser recogido por el GC, permitiendo realizar tareas de limpieza.
- El objeto referenciado se pone en la cola antes de ser eliminado, pero nunca se puede acceder a él a través de la referencia fantasma.
- El **acceso** a la referencia fuerte está asegurado, aunque se elimine, se realiza a través de la cola con `poll`. `get` siempre devuelve `null`, las referencias fantasmas están pensadas para acceder a la memoria cuando las referencias fuertes no están disponibles

Usos principales:

- **Referencias débiles:** para acceder dinámicamente a la referencia fuerte de un objeto hasta que este ya no esté disponible, evitando crear referencias indiscriminadas (aliasing).
- **Referencias suaves y fantasmas:** para hacer **cachés en memoria**, de manera que se pueda acceder a referencias fuertes que ya no están disponibles en memoria.

2.6 Clase String

Los objetos que son cadenas de texto se pueden crear de dos formas diferentes:

- **Directamente:** asignando una cadena de texto al objeto, en cuyo caso se almacenan en una zona del montón llamada **String Pool**, de modo que **cada vez que se asigna la misma cadena de texto, se apunta a la dirección que la contiene**
- **Indirectamente:** el objeto se crea usando un constructor de string, en cuyo caso se almacenan en el montón, pero **fuera del String Pool**, aunque tenga el mismo valor que una cadena previa.

String es una clase de Java, pero **no se comporta** como el resto de las clases en lo que respecta a la asignación entre objetos cuando se usan directamente cadenas de texto. Una cadena de texto es **un objeto inmutable**, es decir, una vez se ha reservado memoria y se le asigna un valor dado, no se puede modificar. El uso indiscriminado de cadenas de texto puede **disminuir mucho el rendimiento**.

Si se van a realizar múltiples modificaciones sobre una cadena de texto, no se debería de utilizar la clase `String`. Se debería usar la clase `StringBuffer` si se vana usar múltiples operaciones sobre cadenas de texto, ya que no se reserva espacio de memoria cada vez que se genera una cadena de texto.

2.7 Método Equals

- Dos objetos son iguales si ocupan la **misma posición de memoria**. Esto es una condición muy **restrictiva**, ya que podrían existir dos objetos con los mismo atributos en posiciones de memoria distintas.

```
Coche coche1 = new Coche();
Coche coche2 = coche1; // coche1 y coche2 apuntan al mismo objeto
System.out.println(coche1.equals(coche2)); // true
```

- Dos objetos son iguales si son del **mismo tipo** y si los valores de todos sus **atributos** también son **iguales**. Eso obliga a que los dos objetos tengan los mismos valores de los atributos en el momento de la comparación y que los atributos sean inmutables.

`equals` es un método que indica si un objeto (el argumento) es igual a otro (desde el que se invoca). Todas las clases lo tienen, pues es heredado de la clase `Object`.

- La implementación heredada sigue la primera opción, ya que lo único que se conoce de cualquier objeto es su referencia.
- Se debe **reimplementar** comparando atributos inmutables, de modo que dos objetos son iguales si estos atributos tienen el mismo valor.

```
@Override
public boolean equals(Object obj) {
    if (this == obj) {
        return true; // Mismo objeto en memoria
    }
    if (obj == null || getClass() != obj.getClass()) {
        return false; // Distinto tipo de objeto o null
    }
    Coche coche = (Coche) obj;
    return this.matricula.equals(coche.matricula);
}
```

[! Buenas Prácticas]

- Todos los objetos que realicen comparaciones deben tener reimplementado el método ``equals`
- Los atributos **inmutables** pueden ser **tipos primitivos, wrappers y strings**.

3. Conjuntos de datos

Escrito por **Adrián Quiroga Linares**.

3.1 Conjuntos de Datos

- Una buena parte de la programación consiste en **manejar conjuntos de datos** sobre los que se realizan operaciones CRUD (creación, lectura, actualización y eliminación).
- Típicamente la mayoría de los lenguajes de programación soportan la representación de conjuntos de datos a través del concepto de **array**. Sin embargo tienen muchas **limitaciones**.
- Se han propuesto una buena cantidad de formas de representar conjunto de datos que intentan dar respuesta a esas limitaciones: arrays, colecciones, iteradores, listas, conjuntos... **Todos los tipos de conjuntos de datos son mapas, colecciones o iteradores**, el resto particulariza el tipo del cual hereda.

3.2 Arrays

Un **array** en Java es conceptualmente igual al resto de lenguajes, un conjunto de elementos del **mismo tipo** que ocupan posiciones de **memoria consecutivas** para facilitar su acceso de forma sencilla a través de un **índice**.

En Java, un array es en realidad un **objeto**. Esto significa que al crear un array, se debe reservar memoria explícitamente usando `new`, a menos que se inicialice directamente con valores como en el ejemplo anterior (su clase es `tipodedatos[]`). Además, hereda métodos de la clase `Object` y tiene un atributo público `length` que almacena su longitud.

```
int[] edades = new int[5];    // Crea un array de enteros con 5 elementos,
                              // inicializados en 0
edades[0] = 10;               // Asigna valores a los elementos
edades[1] = 15;
System.out.println(edades.length); // Imprime la longitud del array: 5
```

Es la única estructura que permite almacenar **tipos primitivos**. Pero presenta varias **limitaciones**:

- Los **arrays** tienen un tamaño **fijo**, una vez se llega al límite no se pueden añadir más elementos.
- **No** se pueden **borrar** elementos cuando el array maneja un **tipo primitivo**. Cuando maneja objetos, se pueden eliminar elementos escribiendo `null` en su posición. Como consecuencia, en todos los posteriores recorridos se deberá tener cuidado con estos valores nulos para evitar un error de ejecución.

- Sólo se deben usar array cuando se sabe que el **número de elementos es fijo** y **no** se va a **eliminar** ninguno.
- Sólo se deben usar cuando se invocan **métodos** de otras clases que **devuelvan un array** y convertirlos a otras estructuras sea ineficiente.

3.3 Colecciones

Colecciones: grupos de datos **desordenados** sobre los que se definen operaciones de inserción, borrado y actualización.

3.3.1 Collection

`Collection<E>` es una interfaz que define las operaciones que debe tener una colección que almacena datos de tipo `E`. **No** se puede **reservar memoria** (instanciar) para un objeto de tipo `Collection` porque es una interfaz.

Los elementos de las colecciones están **desordenados**, así que sus elementos **no** se pueden identificar mediante un **índice**.

Métodos de `Collection` más frecuentes:

- `boolean add(E ele):` añade el elemento `ele` a la colección
- `boolean contains(Object obj):` comprueba si el objeto `obj` se encuentra en la colección
- `boolean isEmpty():` indica si la colección está vacía
- `void remove(Object obj):` elimina el objeto `obj` de la colección
- `Iterator<E> iterator():` genera un iterador que se puede usar para recorrer la colección.

NOTA

`contains` y `remove` usan `equals` para comprobar si el objeto está en la colección

Algunas implementaciones referenciadas de la interfaz `Collection` **no soportan todas sus métodos**. Como de todas formas es necesario que los implementen, generan una excepción del tipo `UnsupportedOperationException`. Por ejemplo, la implementación del método `values()` de la clase `HashMap` devuelve una `Collection` para la cual el método `add()` no está soportado. Esta operación en concreto no se puede hacer, porque estaríamos metiendo un elemento sin clave en el `HashMap`, lo que rompe su estructura

Una de las formas de recorrer todos los elementos de una `Collection` es a través de un **bucle** `for-each`. No se puede hacer un `for` normal porque la colección no tiene índices que recorrer. Sin

embargo surge un **problema**, las colecciones **no se pueden modificar** mientras se están recorriendo porque esto puede hacer que el iterador que la está recorriendo pierda la pista del estado de la colección. Si se intenta, se genera una excepción del tipo `ConcurrentModificationException`

```
Collection<String> nombres = new ArrayList<>();
nombres.add("Ana"); // Añadir un elemento
nombres.add("Juan");

System.out.println(nombres.contains("Ana")); // true
System.out.println(nombres.isEmpty()); // false

nombres.remove("Ana"); // Eliminar un elemento
System.out.println(nombres.contains("Ana")); // false
```

3.3.2 Iterator

`Iterator<E>`: interfaz que define las operaciones que se pueden usar para recorrer los elementos de una colección de datos de tipo `E`. **No** se puede **reservar memoria** (instanciar) para un objeto de tipo `Iterator`, porque es una interfaz.

Métodos de `Iterator`:

- `boolean hasNext()`: indica si existe un elemento en la siguiente posición en la que se encuentra el puntero
- `E next()`: actualiza el puntero a la posición siguiente y obtiene el elemento que se encuentra ahí.
- `remove()`: elimina el elemento que se encuentra en la posición actual

Como los punteros sólo se mueven hacia adelante, un iterador **sólo se puede usar una vez** para recorrer los elementos de la colección, ya que después el puntero del iterador estará en el final de la colección y `hasNext()` devolverá siempre falso.

```
Iterator<String> iterador = nombres.iterator();
while (iterador.hasNext()) {
    String nombre = iterador.next();
    System.out.println(nombre);
    if (nombre.equals("Juan")) {
        iterador.remove(); // Elimina el elemento actual
    }
}
```

Un iterador es una forma alternativa al bucle `for-each` para **recorrer los elementos de una colección**.

- A diferencia de **for-each**, un **iterador permite la eliminación de los elementos mientras se recorre la colección**, eliminando tanto los elementos de la colección como del iterador.
- El **rendimiento de un iterador y de un `for each` es el mismo**, ya que en realidad el compilador interpreta el `for-each` como si fuese un iterador.

3.4 Listas

3.4.1 Listas

`List <E>`: subinterfaz de `Collection`. Además de soportar las operaciones de `Collection`, soporta las que debe tener una lista. **No** se puede **reservar memoria** (instanciar) para un objeto de tipo `List` porque es una interfaz.

Los elementos de las `List` tienen un **orden** que está relacionado con la secuencia en que dichos elementos han sido introducidos durante la ejecución. Este orden permite identificar cada elemento con un **índice** que indica su posición en la `List`. Pueden contener **elementos duplicados**.

Métodos de `List`:

- `E get(int i)`: obtiene el elemento de índice `i`. (`get(null)` no da fallo, devuelve `null`)
- `void set(int i, E ele)`: actualiza el objeto de índice `i` con valor `ele`
- `E remove(int i)`: Elimina el objeto de índice `i`

Las `List` se pueden **recorrer** tanto usando sus **índices**, ya que son ordenadas, como usando un **bucle `for-each`**, ya que son `Collections`.

```
List<String> frutas = new ArrayList<>();
frutas.add("Manzana");
frutas.add("Pera");
frutas.add("Manzana"); // Las listas permiten duplicados

System.out.println(frutas.get(1)); // Pera

frutas.set(1, "Banana"); // Cambia el elemento en la posición 1
System.out.println(frutas.get(1)); // Banana

frutas.remove(0); // Elimina el primer elemento (Manzana)
System.out.println(frutas); // [Banana, Manzana]
```

3.4.2 ArrayList

`ArrayList<E>`: clase que implementa la interfaz `List`. **Sí** se puede **reservar memoria** (instanciar) para un objeto de tipo `ArrayList`, pues es una clase.

Tiene un **atributo público**, `size`. Las **operaciones de acceso** a sus elementos tienen complejidad **lineal**. Además permite introducir **valores nulos**.

El **constructor sin argumentos** de `ArrayList` reserva espacio para 10 elementos. `ArrayList` realiza una **redimensión dinámica** cuando el número de datos que se almacena es **mayor que la capacidad** de almacenamiento de la lista. Por defecto, se aumentará la capacidad en 10. Si la redimensión se realiza **frecuentemente** se penalizará el **rendimiento** de la lista, ya que internamente la redimensión supone incrementar el tamaño de un array normal.

```
ArrayList<Tipo> nombre = new ArrayList<>();
ArrayList<Tipo> nombre = new ArrayList<Tipo>();
ArrayList<Tipo> nombre = new ArrayList<>(capacidad);
ArrayList<Tipo> nombre = new ArrayList<>(coleccion);
ArrayList nombre = new ArrayList();
ArrayList<Tipo> nombre = new ArrayList<>(){};
ArrayList<Tipo> nombre = new ArrayList<>(){{ add(...); }};
```

Los `ArrayList` se **recorren** usando un bucle `for` normal. Al **eliminar** elementos durante el recorrido habrá que tener cuidado con que el **índice no supere el nuevo tamaño**. Durante el recorrido hay que comprobar que el elemento actual es **distinto** de `null` antes de operar sobre él.

En realidad, la clase `ArrayList` encapsula un **array normal** de objetos al que se accede a través de los métodos de la interfaz `List`.

- La **pila** almacena la **referencia al array de objetos**
- El **montón** almacena el **array de objetos** y los **demás atributos** de `ArrayList` (`size`)

```
ArrayList<Integer> numeros = new ArrayList<>(); // Crea un ArrayList con capacidad
para 10 elementos
for (int i = 1; i <= 15; i++) {
    numeros.add(i); // Al llegar a 11 elementos, el ArrayList se redimensionará
    automáticamente
}

System.out.prreferenciadointln("Tamaño: " + numeros.size()); // 15
System.out.println("Elemento en la posición 10: " + numeros.get(10)); // 11
```

```
numeros.set(5, 100); // Actualiza el elemento en la posición 5
System.out.println(numeros); // Muestra todos los elementos
```

3.5 Conjuntos

Conjuntos: colecciones en las que **no se repiten los datos**. Eso significa que los datos serán diferentes de acuerdo al criterio de igualdad definido por ellos (método `equals`).

3.5.1 Set

`Set<E>`: subinterfaz de `Collection` que soporta exactamente las mismas operaciones que una `Collection`. **No** se puede **reservar memoria** (instanciar) para un objeto de tipo `Set`, pues es una interfaz.

Los elementos del conjunto **son únicos**. Para asegurarse de esto, **se debe sobrescribir el método `equals` de la clase `E`**. Se debe controlar que la **modificación** de los objetos no se dé como resultado que dicho objeto sea igual a otro de los almacenados, porque se generarían inconsistencias cuando las modificaciones **no se realizan con los métodos de `Set`**, si no que se realizan por aliasing. Como consecuencia, **permiten sólo un valor nulo**.

Como es una colección, se **recorre** con un **bucle** `for-each` o un **iterador**, no con un índice.

3.5.2 HashSet

`HashSet<E>`: clase que implementa la interfaz `Set`. **Sí** se puede **reservar memoria** para un objeto de tipo `HashSet`, porque es una clase.

Internamente, sus datos se almacenan como las **claves de un objeto `HashMap`** (se insertan en base al **hashCode**).

- Sus valores están **desordenados**, pues se almacenan en una **tabla hash** de acuerdo a una **función hash**.
- Las operaciones de **acceso** a sus elementos se realizan en tiempo constante $O(1)$

```
Set<Integer> numeros = new HashSet<>();
numeros.add(10);
numeros.add(20);
numeros.add(10); // Este 10 no se añadirá porque ya existe en el conjunto

System.out.println(numeros); // Muestra los elementos en un orden basado en su hash
code
```



```

numeros.add(null); // Se permite añadir un único elemento null
System.out.println(numeros.contains(null)); // true

// Recorrido del HashSet
for (Integer numero : numeros) {
    System.out.println(numero);
}

```

3.6 Mapas

3.6.1 Map

`Map <K,V>`: interfaz que relaciona una clave con un valor de manera que **a partir de la clave se obtiene el valor asociado a ella**. No se puede **reservar memoria** para un objeto de tipo Map, porque es una interfaz.

Las **claves no pueden estar repetidas**, pero los **objetos sí** (por tanto varias claves pueden tener asociado un mismo valor). Tanto las claves como los valores tienen que ser **objetos**, no pueden ser **tipos primitivos**. Algunas implementaciones permiten almacenar **claves y valores nulos**.

La **clave** puede ser **cualquier objeto**, siempre que su clase siga las siguientes **reglas**:

- El `equals()` **debe estar sobrescrito**, pues se usará para localizar y obtener la clave con la que recupera un valor.
- El objeto clave debe ser **inmutable** (es decir, no cambiará nunca una vez se realiza la reserva de memoria) pues si se cambiara, perderíamos el acceso al valor asociado a la clave, pues el identificador de la entrada (`hashCode()`) se calculó en base al valor de la clave. Si la clave no es inmutable, entonces es necesario controlar que **no se modifiquen aquellos atributos usados en `equals()`** para comprobar la igualdad entre objetos.

Métodos de `Map`:

- `V get(Object key)`: Devuelve el valor asociado a la clave `key`
- `V put(K key, V value)`: Asocia un valor `value` al que se le asocia una clave `key`
- `V remove(Object key)`: Elimina el par clave-valor basado en la clave.
- `boolean containsValue(Object value)`: indica si el valor `value` está almacenado en el mapa, es decir, si tiene asociada una clave.
- `boolean containsKey(Object key)`: indica si existe algún dato asociado a la clave `key`
- `size()`

`containsKey()` **es más eficiente** que `containsValue()` porque aprovecha la estructura de hashing de la implementación del `Map` para realizar la búsqueda de claves.

Algunas implementaciones de la interfaz `Map` **no soportan todos sus métodos**. Como de todas formas es necesario que los implementen, generan una excepción del tipo `UnsupportedOperationException`

Los objetos almacenados en un `Map` **no se pueden recorrer directamente**, ya que su objetivo es acceder a los valores a partir de las claves, no realiza una búsqueda de datos usando un criterio diferente de las claves. `Map` define métodos para convertir las claves y **valores en estructuras de datos que se pueden recorrer**, lo cual facilita el acceso a cada componente del mapa:

- `Collection<V> values()`: Devuelve una **colección** con todos los **valores** del mapa, porque no se conoce el orden en el que se insertaron
- `Set<K> keySet()`: Devuelve un **conjunto** con todas las claves, porque no puede haber claves duplicadas
- `Set<Map.Entry<K, V>> entrySet()`: Devuelve un conjunto con todas las entradas `<clave, valor>`. Devuelve un **conjuntos** porque no puede haber duplas duplicadas.

También se puede usar `Iterator<Map.Entry<String, Contiente>> entrySet()`

Map Entry

`Map.Entry<K, V>`: Interfaz que define métodos para acceder a una entrada de un mapa

- `K getKey()`: permite acceder a la clave de la entrada en el mapa
- `V getValue()`: permite acceder al valor de la entrada en el mapa

3.6.2 HashMap

`HashMap<K, V>`: clase que implementa la interfaz `Map`. **Sí** se puede **reservar memoria** para un objeto de tipo `HashMap`, pues es una clase.

Los valores del `HashMap` están **desordenados**, pues se almacenan en una **tabla hash** de acuerdo con una **función hash**. Sin embargo, las operaciones de **acceso** a las entradas del `HashMap` se realizan en tiempo constante $O(1)$

Cada clave del mapa está asociada a un **código hash (hashcode)**, que es un entero generado aleatoriamente a partir de una semilla. Se supone que los `hashCode` son **únicos** para cada objeto, pero como son generados aleatoriamente es **imposible asegurar esto**. Por tanto, **cuando se comparan dos entradas** de un `HashMap`

- Se comparan sus `hashCode`: si son distintos, se entiende que los objetos son distintos, si son iguales se pasa al paso 2
- Se invoca a `equals()` para aplicar el criterio de igualdad.

El uso de los `hashCode` simplifica y hace más **eficiente el acceso** a los datos en un `HashMap` pues en primer lugar compran enteros y sólo ejecutará `equals()` (que puede tener una implementación con comparaciones muy complejas) cuando coincidan.

Para **generar** el `hashCode`, es necesario **sobreescribir** el método `hashCode()` heredado de `Object` usando el valor de los **atributos inmutables de la clave** como semillas. La implementación de `hashCode()` en `Object` es un método nativo escrito en C. Se generan `hashCodes` parciales usando estas semillas en el método `hashCode()` de la clase `Objects` (el de `Object` no funciona con valores nulos).

```
HashMap<K,V> m = new HashMap<>();
HashMap<K,V> m = new HashMap<K,V>();
HashMap<K,V> m = new HashMap<>(capacidad);
HashMap<K,V> m = new HashMap<>(capacidad, factorDeCarga);
HashMap<K,V> m = new HashMap<>(otroMapa);
HashMap m = new HashMap();
HashMap<K,V> m = new HashMap<>({});
HashMap<K,V> m = new HashMap<>(){ put(..., ...); };
```

3.7 Comparativa

Según las **características** de los conjuntos de datos escogeremos uno y otro en función de para qué lo necesitamos:

Las **listas** ocupan **menos memoria** que los mapas, y los **mapas** son **más rápidos** que las listas. Por lo general:

- Si hay **restricciones de memoria** respecto a la cantidad de datos a almacenar y la **eficiencia no es la prioridad**: `ArrayList`
- Si **se necesita acceder en el orden de llegada**: `ArrayList`
- Si **no se necesita acceder** a los datos **en el orden en el que se almacenaron** y **se dispone de un identificador unívoco**: `HashMap`
- Si **no se necesita acceder** a los datos **en el orden en el que se almacenaron** y los datos **no se pueden repetir**: `HashSet`

4. Herencia

Escrito por **Adrián Quiroga Linares**.

4.1 Concepto de Herencia

La **herencia** es el mecanismo por el cual una **clase derivada** reutiliza los atributos y métodos de una **clase base** o superior. Una **clase derivada** se considera una **extensión de la clase base**. Sus métodos y atributos son:

- Aquellos heredados de la clase base, que son copiados **implícitamente**. Los constructores de la clase base no se heredan, ya que se consideran específicos a ella.
- Aquellos definidos **explícitamente** como propios de la clase derivada. Se dice que hay una relación del tipo "**es un/a**" entre una clase derivada y una clase base

4.1.1 Tipos de Herencia

- **Herencia simple:** la clase B hereda los atributos y métodos de la clase A
- **Herencia multinivel:** la clase C hereda los atributos y métodos de la clase B que, a su vez, hereda los atributos y métodos de la clase A . Por tanto, C hereda también los atributos y métodos de la clase A
- **Herencia jerárquica:** las clases B y C heredan los atributos y métodos de la clase A , de modo que se diferencian por sus atributos y métodos propios.
- **Herencia múltiple:** la clase C hereda los atributos y métodos de las clases A y B , por lo que hará falta definir **políticas de herencia** si A y B tienen métodos comunes.

4.1.2 Beneficios de la Herencia

El principal beneficio es la **reutilización de código**, pues un segmento de código que ya ha sido desarrollado, depurado y validado en una clase se usa en otro sin tener que cambiarlo. Esto **simplifica el código**, porque evita tener que implementar varias veces el mismo método.

En principio, **facilita el mantenimiento de los programas**, porque el cambio o reparación de los métodos se realiza sólo en las clases base, y no en todas las que los usan. En principio, **facilita la extensibilidad de los programas**, porque permite construir nuevas clases directamente a partir de otras que ya existían, en vez de empezar de cero.

4.1.3 Desventajas de la Herencia

Puede ir en contra del concepto de de encapsulación, pues para facilitar su uso, se puede precisar que los atributos tengan un modificador de acceso diferente al privado. **Puede complicar el mantenimiento de los programas**, pues los cambios en las clases base tienen un impacto en las derivadas, por lo que estos cambios podrían ser **inconsistentes** con el código específico de las clases derivadas **Puede dificultar la extensibilidad de los programas** cuando la jerarquización es muy profunda, pues el código será mucho más **difícil de entender** ya que las relaciones entre clases serán complicadas y será difícil identificar a qué clase pertenece cada método.

4.2 Concepto de composición

La **composición** es el mecanismo por el cual una clase contiene **objetos de otras clases**, a los que se les **delegan** ciertas operaciones para conseguir la funcionalidad buscada. Se dice que hay una relación del tipo "**tiene un/a**" (o "**tiene los atributos de un/a**") entre las clases que participan en una composición.

4.2.1 Beneficios de la Composición

- El principal beneficio es la **repartición de responsabilidades entre objetos**, pues cada uno se encarga de realizar determinadas operaciones, que después podrán invocar otros.
- **Facilita el mantenimiento de los programas**, pues las clases están más **desacopladas entre sí**, por lo que no comparten métodos, así que es poco probable que cambiar algo en una clase cree una inconsistencia en otra.
- **Facilita la extensibilidad de los programas**, pues permite implementar funcionalidad en clases usando otras que ya existían.

4.2.2 Desventajas de usar Composición

- **Genera mucho más código y mucho más complejo** para hacer lo mismo que consigue la herencia.
- **Es más lento de desarrollar** que la herencia, pues la construcción de nuevas clases se realiza desde cero.

4.3 Herencia contra Composición

Debemos escoger composición sobre herencia cuando:

- Las clases **no están relacionadas lógicamente**: la herencia no tendría sentido
- La clase base tendría **una única derivada**: la herencia no tendría sentido
- Las clases derivadas heredarían **código no necesario**: la herencia no tendría sentido

- Existe la posibilidad de que las **clases base cambien**: la herencia complicaría el desarrollo
- Las clases derivadas tendrían que **sobreescibir** muchos métodos: la herencia complicaría el desarrollo.

4.3 Herencia en Java

En java se **imponen restricciones** sobre la herencia:

- **No existe herencia múltiple**
- Los atributos y métodos se heredan en función del **tipo de acceso** que tengan en la clase.

4.3.1 Revisitando el tipo de Acceso

- ``private:`` la clase derivada ****nunca hereda**** el atributo/método - ``public:`` la clase derivada ****siempre hereda**** el atributo/método - ``acceso a paquete:`` la clase derivada ****solo hereda**** el atributo/método si se encuentra en el ****mismo paquete**** que la clase base - ``protected:`` la clase derivada ****siempre hereda**** el atributo/método

Como los atributos privados no se heredan, **se podrían hacer públicos** o protegidos. Esto eliminaría (si son públicos) o debilitaría (si son protegidos) la encapsulación. Por tanto, los atributos se **mantienen privados**, ya que se considera que la **encapsulación tiene más beneficios que la herencia**:

- Sin encapsulación, el desarrollo, mantenimiento y validación de los programas sería mucho más difícil.
- Sin encapsulación, la composición no tiene sentido.

[!Buenas Prácticas]

- Los **atributos** son **siempre privados**, aunque se use herencia
- En una jerarquía de clases, se debería implementar los **métodos de la clase donde se definen los atributos que usan**, y no en sus derivadas.

Entonces, la clase derivada **no** hereda los **atributos** ni los **métodos privados**, pero **sí** los **métodos públicos**, lo cual suscita ciertas preguntas:

- ¿Cómo va a ejecutar los métodos heredados si no hereda los atributos que usan?
 - En realidad, la clase derivada **hereda todos los atributos** de la clase, independientemente de su tipo de acceso.
 - El tipo de acceso de un atributo especifica su nivel de visibilidad. Por tanto, los atributos privados se heredan, pero no se puede acceder a ellos directamente, si no que para ello se usan métodos.

- ¿Cómo va a ejecutar los métodos heredados si no heredo los métodos que usan?
 - Igual que para los atributos, en realizada la clase derivada hereda todos los métodos, independientemente del tipo de acceso.
 - El tipo de acceso de un método especifica su **nivel de visibilidad**. Por tanto, los métodos privados se heredan, pero no se puede acceder a ellos directamente, si no que para ellos se usan los métodos heredados.

4.3.2 Constructores

Como se dijo antes, los **constructores** de la clase **no se heredan**, ya que se consideran **específicos a ella**. Los constructores de una clase reservan memoria e inician los atributos de esa clase, pero el constructor de la clase base no tiene acceso a los atributos de la derivada. Por tanto, si se pudiesen heredar los constructores de la clase base, no se podrían invocar para crear objetos de la clase derivada.

¿Cómo consiguen los constructores de la clase derivada reservar memoria e inicializar los atributos privados de la clase base?

- En los constructores de la clase derivada **sin argumentos**:
 - Al invocarlos, se invoca **automáticamente al constructor sin argumentos de la clase base**.
 - Por tanto, la clase base debe tener **obligatoriamente** un constructor sin argumentos. En caso contrario, se generará un **error de compilación**.
 - Si la clase base tiene algún constructor explícito, uno de ellos tendrá que ser sin argumentos.
 - Si la clase base no tiene ningún constructor explícito, el constructor por defecto es suficiente.
- En los constructores de la clase derivada **con argumentos**:
 - **No se invoca automáticamente** a ningún **constructor de la clase base**, ya que no se puede garantizar que exista alguno con los mismos argumentos.
 - Por tanto, habrá que invocar **manualmente** el constructor de la clase base que se considere oportuno. Para poder hacer esto necesitamos `super(args)`, que es un método que nos permite acceder desde la clase derivada a los atributos, métodos y constructores de la clase base que tengan cierto nivel de visibilidad.
 - `super(args)` sólo nos permite acceder a los elementos de la clase base inmediatamente superior a la clase derivada donde se invoca.

Java debe **crear un objeto completo de tipo Infanteria**, pero ese objeto también es, internamente, un objeto de tipo CartaDeEquipamiento (su clase base).

1. **Reserva toda la memoria** para el objeto completo (Infanteria + lo heredado de CartaDeEquipamiento).
2. **Inicializa primero la parte de la clase base (CartaDeEquipamiento):**

- Esto incluye inicializar todos los atributos definidos en la clase base.
- Para eso, se llama primero al constructor de la clase base.

3. Luego ejecuta el constructor de la subclase (Infanteria):

- Ahora puede inicializar los atributos y comportamientos propios de la subclase.

Necesidad de construir desde lo más general a lo más específico: - La subclase depende de que la clase base esté correctamente inicializada (por ejemplo, que los atributos privados de la base estén listos).

4.4 Sobreescritura de Métodos

La **sobreescritura de métodos** es un mecanismo mediante el cual un método heredado de una clase base vuelve a ser implementado de manera distinta en la clase derivada. Para sobrescribir un método, se usa la palabra clave `@Override` sobre la reimplementación en la clase derivada.

- El **nombre** del método, sus **argumentos** y el **tipo que devuelve** deben ser los mismos en la clase base y en la derivada.
- El **tipo de acceso** del método en la clase derivada debe ser igual o superior al que tiene en la clase base.
- Un método se sobrescribe cuando la implementación de la clase base no es válida para la derivada o es conveniente adaptar la implementación de la clase base a las características específicas de la derivada.

Se puede aprovechar la implementación original de un método en su sobreescritura usando `super()`. Al llamar al método `super()` en la sobreescritura en la clase derivada, se invoca la implementación de la clase base del método que se está sobrescribiendo. Se usa `super()` y no se invoca directamente el método pues la clase derivada no sabría distinguir si nos referimos a la sobreescritura o la implementación de la clase base.

Todas las clases creadas en Java son derivadas de `Object`, por lo que heredan todos sus métodos:

- `getClass:` indica la clase a la que pertenece el objeto que lo invoca
- `notify` y `wait:` orientados a la gestión de los hilos
- `finalize:` se invoca cuando el recolector de basura elimina el objeto de la memoria del programa. Es necesario sobrescribirlo.

4.5 Clases Abstractas

Las **clase abstractas** son clases que no se pueden instanciar.

- Pueden tener **constructores**. No se pueden invocar a través de `new`, sólo se pueden invocar mediante `super()` desde una clase derivada

- Pueden tener **atributos**
- Pueden tener **métodos implementados** y métodos **abstractos** Una clase debería ser abstracta cuando no es necesario que tenga objetos pues todos los objetos pertenecen a cualquiera de las otras clases de la jerarquía

Se suelen usar como **clases base** de otras clases que sí son instanciables, ocupando los primeros niveles de la jerarquía de clases. Por tanto, deben tener **implementados** la mayor cantidad posible de métodos para que las clases derivadas puedan heredarlos y así favorecer la **reutilización de código**.

Una clase abstracta:

- Puede tener atributos de cualquier tipo
- Puede tener todos los métodos implementados
- Puede tener todos los métodos abstractos, debería tener tantos métodos implementados como sea posible
 - Si todos sus métodos son abstractos, el enfoque del diseño será que todas sus derivadas tengan los mismo métodos que ella. Así las clases son más independientes, lo que facilita el mantenimiento y la extensibilidad del programa
 - Si implementa tantos métodos como sea posible, el enfoque del diseño, será la reutilización de código.
- Puede no tener constructores, debería tener tantos constructores como sea necesario.
- Puede ocupar cualquier lugar de la jerarquía de clases, debería ocupar los niveles superiores
- Puede ser una derivada de una clase no abstracta, debería ser derivada de otra clase abstracta

NOTA

Si tenemos una clase abstracta con métodos abstractos que hereda de otra clase abstracta con métodos abstractos, no hace falta escribir de nuevo los métodos abstractos en la derivada a no ser que queramos especificar su comportamiento ahí.

4.5.1 Métodos abstractos

Los **métodos abstractos** son métodos que no tienen cuerpo, es decir, no están implementados. Sólomente se especifica su **nombre**, el **tipo de datos** que devuelve y sus **argumentos**. `public abstract tipo_dato nombre_metodo(tipo_dato2 arg);` Todas las clases que contengan métodos abstractos son **clases abstractas**. Los métodos abstractos tienen que estar **implementados** en **todas las clases derivadas** de la clase abstracta a la que pertenecen siempre que estas **no sean abstractas**.

4.6 Clases, Atributos y Métodos Finales

Las **clases finales** son clases a partir de las cuales no se pueden crear clases derivadas, es decir, son los últimos nodos de la jerarquía de clases. Una clase debería ser final si se puede garantizar que no

será necesario crear clases derivadas de ella.

Los **métodos finales** son métodos que no se pueden sobrescribir. Un método debería ser final si se puede garantizar que no será necesario realizar modificaciones sobre él en las clases derivadas de la clase donde se implementa.

Los **atributos finales** son atributos cuyo valor, una vez establecido, no se puede modificar. No tienen setters. Se usan para implementar **constantes de los programas**. `public final static tipo_dato nombre_atributo=valor;` `nombre_clase.nombre_atributo`

Se suelen definir en **clases abstractas** accesibles por todas las clases que las usan. Se suelen establecer como `static` para poder usarlas sin tener que crear un objeto de la clase en la que están definidas. La palabra clave `static` para atributos o métodos hace que se almacenen en la memoria estática, permitiendo que estén disponibles desde el inicio del programa sin tener que instanciar su clase.

IMPORTANTE

Qué se hereda?? Los **atributos** y **métodos** independientemente de su **tipo de acceso** (*esto solo afecta a su nivel de visibilidad*). Incluyendo los **final**. **Qué no se hereda??** Los métodos y atributos **static en una interfaz o en clases**. (*Manolo Lama Said*)

5. Polimorfismo

Escrito por **Adrián Quiroga Linares**.

5.1 Polimorfismo en herencia

El **polimorfismo** en herencia es el mecanismo por el cual un objeto se puede **comportar de múltiples formas** en cada momento, en función del **contexto** en el que aparece en el programa. Un objeto se puede comportar como una instancia de:

- La clase a la que **pertenece**, aquella cuyo constructor se invocó a través de `new()`
- Una de las clases que se encuentran en un **nivel superior** a la clase a la que pertenece. Esto incluye **clases abstractas**, pues la restricción impuesta sobre ellas es que no se pueden instanciar (usar `new()` para invocar sus constructores), pero en este caso el objeto se estaría inicializando usando constructores de otras clases. Que un objeto se comporte como una de sus clases base abstracta sirve para restringir los métodos que puede invocar.

5.2 Upcasting y Downcasting

Cuando se hace que un objeto se comporte como una clase distinta a la que pertenece, está forzando un **casting**, un cambio de tipo de dato en él. El casting **no implica una nueva reserva** de memoria: el objeto sigue siendo el mismo, independientemente de los castings que se hagan sobre él. El casting **afecta a la visibilidad** de los atributos y métodos que se pueden invocar desde el objeto.

Los únicos métodos y atributos accesibles por el objeto al que se le ha hecho casting son:

- Aquellos definidos explícitamente en la clase a la cual se ha realizado el cast
- Aquellos que hereda (y tiene acceso) la clase a la que se hizo el cast

5.2.1 Upcasting

En **upcasting** un objeto de una clase derivada se comporta como su clase base en la jerarquía. Cuando se realiza un upcasting no es necesario indicar de forma explícita a qué clase base se realiza el cast, pues sólo hay una.

```
Animal a = new Perro();
```

 upcasting automático y seguro

Es el mecanismo de polimorfismo más **habitual**, ya que el natural pensar que un objeto de una clase derivada se comporta como su clase base. Es una **concepto clave en la POO**, porque facilita la toma

de decisiones respecto al diseño del programa: si se necesita usar upcasting, entonces se deberá usar herencia.

NOTA

Si un método está sobrescrito cuando se realiza upcasting, el objeto usará la implementación del método correspondiente a la clase a la que pertenece el objeto, de modo que aunque se comporta como otra clase, no utiliza las implementaciones de los métodos de esa otra clase

5.2.2 Downcasting

En **downcasting** un objeto de una clase base se comporta como una de sus clases derivadas de la jerarquía. Cuando se realiza un downcasting es necesario indicar de forma explícita a qué clase derivada se realiza el cast

```
ClaseDerivada nombreObjDerivado = (ClaseDerivada) nombreObjBase;
```

Normalmente, sólo se usa cuando se necesita **deshacer el resultado de un upcasting** anterior, pues **fuerza el comportamiento** de los objetos, ya que normalmente no se puede asegurar que un objeto de una clase base se comporta como una de sus clases derivadas.

Es una operación no segura (`unsafe`), ya que puede generar una excepción en tiempo de ejecución (`ClassCastException`) pues no se puede garantizar que el objeto disponga de todos los métodos y atributos de la clase a la que se castea.

- En tiempo de diseño, el compilador no comprueba si el cast se realiza correctamente, ni siquiera comprueba si ambas clases pertenecen a la misma jerarquía
- En tiempo de ejecución tiene lugar el casteo, pero sólo se comprueba si el objeto pertenece a la clase derivada a la que se castea. Aunque la clase a la que pertenece el objeto soporte los mismos métodos y atributos que la clase derivada, si no lo es, se generará la excepción.
- Por tanto, se debe **comprobar en tiempo de ejecución** que la clase a la que pertenece el objeto es la clase a la que se realiza en casting usando el operador `instanceof` `<nombre_objeto>`
`instanceof <nombre_paquete>.<nombre_clase>`

5.3 Beneficios del Polimorfismo

Facilita la **simplificación del código** ya que, como los objetos se pueden manejar como objetos de su clase base, la invocación de los métodos comunes a las clases derivadas se realiza desde una sola clase (la clase base).

Facilita la **extensibilidad de los programas** ya que incluir nuevas clases que implementen los métodos comunes de las clases base no supondrá modificaciones en el código. Eso no se cumple con

downcasting, ya que incluir nuevas clases puede implicar cambios en el código, pero serían simplemente extensiones en los métodos en los que ya se manejan las distintas clases derivadas de la clase base.

[!Buenas Prácticas]

- En la medida de lo posible se debe favorecer el uso del polimorfismo pues sus beneficios son muy grandes
- En los conjuntos de datos (como `ArrayList` o `HashMap`) se deben usar las clases derivadas para almacenar los objetos.

6. Interfaces

Escrito por **Adrián Quiroga Linares**.

6.1 Concepto de Interfaz

Una **interfaz** se entiende como un compromiso entre los programadores de una aplicación para que las clases que crean unos puedan ser usadas por otros sin errores o necesidad de adaptarlas. En las interfaces se define:

- Los **tipos de datos** que se deben crear en el programa. En el programa se pueden definir otras, pero al menos deben estar definidos los indicados en las interfaces.
- Los **métodos** que deben tener las clases del programa, indicando exactamente sus nombres, tipo de dato de los argumentos que recibe y el tipo de dato que devuelven. En las clases se pueden definir otras, pero al menos deben tener los de las interfaces que implementan.

El objetivo de las interfaces es establecer los **métodos de interés**, que alguna clase debe implementar y que serán visibles y accesibles por las otras clases. Estos métodos serán métodos **públicos**, pero no todos los métodos públicos son de interés, sólo lo son aquellos que proveen la **funcionalidad requerida por las otras clases**.

El uso de interfaces **facilita el desarrollo y mantenimiento** de los programas, ya que con ellas se dividen las responsabilidades entre los programadores y las clases se pueden implementar de manera independiente.

Un cambio en una clase que implementa los métodos de una interfaz no afectará a las otras clases que usan esa interfaz. Un cambio en la interfaz afectará de forma significativa tanto a las que clases que la implementan como a las que la usan. Por tanto, las **interfaces deberían ser constantes**, una vez se definen no se deberían cambiar.

6.2 Interfaces en Java

En java, una interfaz se entiende como una plantilla de una clase. Están formadas por constantes, declaraciones de métodos abstractos, métodos por defecto y métodos estáticos.

Las interfaces **no tienen constructores** porque **no se pueden instanciar**. Los **atributos** de una interfaz son implícitamente **públicos, estáticos y finales** (constantes). Los **métodos** de una interfaz son implícitamente **públicos y abstractos**.

Para que la interfaz se pueda usar, **alguna clase debe implementarla**, es decir, implementar sus métodos abstractos: `public class <nombre_class> implements <nombre_interfaz>`

Una vez la **interfaz** haya sido implementada, **se comportará como una clase**, pudiendo invocar sus métodos abstractos (usando la implementación de la clase que la implementa), estáticos y por defecto.

Cuando una **clase** implementa una interfaz, **adquiere todos sus métodos**, igual que sucede en la herencia de clases. Entonces, se pueden entender que la clase que implementa una interfaz es su "derivada". Como consecuencia, con interfaces se pueden aplicar todos los conceptos de **herencia, jerarquías, clases abstractas y polimorfismo**.

Una interfaz es **funcionalmente equivalente** a (provee la misma funcionalidad que) una clase abstracta en la que todos los métodos son abstractos, todos los atributos son constantes y no tiene constructores.

Las **diferencias** entre clases abstractas e interfaces son:

- Escogeremos **interfaces** cuando queramos establecer de forma clara e inequívoca los métodos que se van a usar en el resto de clases del programa.
- Escogeremos **clases abstractas** cuando queramos hacer énfasis en la reutilización de código, incluyendo constructores que se invocan desde las clases base.

6.3 Métodos por defecto

Los **métodos por defecto** son métodos que se deben declarar e implementar en la interfaz de manera que los heredan las clases que la implementan. `default <tipo> <nombre_método> (<tipo> <nombre_argumento>){codigo}`

Operan únicamente con sus **argumentos**, pues todos los atributos de la interfaz son constantes. Si se quiere operar con los atributos de las clases que la implementan, se tendrán que pasar como argumento. Entonces, en este caso se debe valorar la conveniencia de usar métodos por defecto. El código será más difícil de entender y mantener, pues los métodos por defecto (que las clases que implementan la interfaz heredan) tendrán argumentos que no serían necesarios si se definieran en dicha clase.

Son **implícitamente públicos**. Pueden **usar métodos abstractos de la interfaz**, pues estarán implementados en las clases que la implementan.

Los métodos por defecto se heredan en las clases que implementan la interfaz y en las interfaces derivadas de la interfaz. Puede **sobreescribirse** y en la sobreescritura se puede usar `super` de la siguiente manera: `<nombre_interfaz>.super.<nombre_método>`

6.4 Métodos estáticos

Los **métodos estáticos** son aquellos que pueden ser invocados sin instanciar la clase en la que están definidos.

- Se cargan en memoria al arrancar el programa, así que **están disponibles desde el inicio** del programa.
- Se pueden definir en **interfaces** y en **cualquier tipo de clase** (abstracta, instanciable o final)
- Sólo pueden **invocar métodos** de la clase o interfaz en la que están definidos que también sean **estáticos**. Los métodos **estáticos no son heredados** ni en las clases ni en las interfaces derivadas.

Métodos por defecto	Métodos estáticos
Sólo se pueden definir en interfaces.	Se pueden definir en interfaces y clases.
Pueden invocar cualquier tipo de método, incluyendo métodos por defecto, estáticos y abstractos.	Sólo pueden invocar métodos estáticos.
Son heredados por las clases e interfaces derivadas de la interfaz que los define.	No pueden ser heredados, son específicos de las clases o interfaces que los definen.
No están disponibles desde el arranque del programa, hay que crear un objeto para invocarlos.	Están disponibles desde el arranque del programa.
Ambos son estáticos, por lo que operan sólo sobre sus argumentos.	

6.5 Herencia e Interfaces

Una clase que implementa una interfaz es como su derivada y además puede implementar más de una interfaz, se puede entender en que las interfaces existe **herencia múltiple**.

Cuando una **clase implementa varias interfaces**, estas pueden tener algunos **métodos en común**.

- Si tienen en común métodos abstractos: no hay conflictos, la implementación en la clase será válida para todas las interfaces.
- Si tienen en común métodos estáticos, pues la clase no los hereda
- Si tienen en común métodos **por defecto** hay colisión entre los métodos, pues no existe ningún mecanismo para seleccionar automáticamente qué implementación se hereda. Para solucionar esto, la clase debe sobrescribir los métodos por defecto que son comunes a las interfaces que implementa. Así, en la propia sobreescritura se puede escoger qué implementación se usa con `super` (o se puede hacer otra implementación nueva).

Cuando una **interfaz hereda de varias interfaces** que tienen algún **método por defecto en común**, hay el mismo problema y se soluciona de la misma manera, sobrescribiéndolo.

Entonces se pueden crear **jerarquías de interfaces**, que seguirán las siguientes reglas:

- Una interfaz derivada puede contener varias interfaces base, es decir, hay **herencia múltiple entre interfaces**.

- Como todos los métodos de una interfaz son implícitamente públicos, **las interfaces derivadas heredarán todos los métodos** de las bases, **excepto los estáticos**.
- **No** se puede establecer **herencia entre clases e interfaces**, es decir una clase no puede extender una interfaz ni viceversa.

Entonces, las jerarquías totales combinan los siguientes tipos de relaciones:

- **Herencia de clases**
- **Herencia de interfaces**
- **Implementación de interfaces**

Por tanto pueden ser muy **complejas**, pero lo importante es que son **extensibles** de manera relativamente sencilla

7. Excepciones

Escrito por **Adrián Quiroga Linares**.

7.1 Concepto de Excepción

En la programación de los métodos se debe favorecer la **separación** lógica de los diferentes tipos de código:

- Funcionalidad provista por el método
- Interacción con usuarios u otros programas
- Almacenamiento y acceso a los datos
- **Gestión de errores**

Una **excepción** es la ocurrencia de **errores y/o situaciones de interés** durante la **ejecución** de los métodos que proporcionan la funcionalidad del programa. El mecanismo de **Java** para el tratamiento de las excepciones aplica la filosofía de **separar el código** para la detección, comunicación y tratamiento de excepciones del código que proporciona la funcionalidad del programa.

7.2 Definición de Excepción

Todas las excepciones son una clase derivada de la clase `Throwable`, cuyos métodos más usados son:

- `String getMessage()`: devuelve el mensaje indicado en la ocurrencia de la excepción
- `void printStackTrace()`: imprime por consola la secuencia de invocaciones de métodos que llevó a la ocurrencia de la excepción.

En Java existen dos tipos de excepciones:

- **Excepciones chequeadas:** se comprueba en tiempo de compilación en qué métodos pueden ocurrir. Los métodos en los que puedan ocurrir una determinada excepción chequeada durante su ejecución deben **indicarlo explícitamente**. Son clases derivadas de la clase `Exception`
- **Excepciones no chequeadas:** no se comprueba en tiempo de compilación en qué métodos pueden ocurrir. El programador debe chequear en qué métodos puede ocurrir una determinada excepción no chequeada para intentar capturarla y tratarla. Son clases derivadas de las clases `RuntimeException` y `Error`

Si el error o situación de interés que provoca la ocurrencia de la excepción no impide la ejecución del programa, esta debería ser chequeada.

La definición de excepciones como clases derivadas de `Exception` le da al programador más control sobre su gestión. Estas clases se instancia en el momento en el que ocurre la excepción. Sus constructores deben invocar el constructor `Exception(String message)`, en el que `message` es el mensaje con el que el programador suele explicar por qué ha ocurrido la excepción para mostrárselo al usuario.

Los métodos que puedan provocar ocurrencias de excepciones se deben **etiquetar** para indicar explícitamente qué excepciones pueden provocar. Un mismo método puede estar etiquetado con más de una excepción, es decir, más de una subclase de `Exception`. Si todas las excepciones que podría provocar un método pertenecen a la misma jerarquía, sería suficiente etiquetar el método con su clase base o clases superiores a ella.

```
<tipo_acceso> <tipo_return> metodo (<args>) throws <excepcion1>, <excepcion2>, ...  
<tipo_acceso> clase (<args>) throws <excepcion1>, <excepcion2>,...
```

Los constructores también pueden provocar excepciones.

7.3 Gestión de Excepciones: Intentar

El bloque `try{}` indica la parte del código en el que un *método_A* **intenta ejecutar con éxito** un *método_B* que puede provocar alguna excepción.

En el código del *método_A* puede haber tantos bloques `try{}` como invocaciones a métodos que pueden provocar una excepción, incluso aunque sean varias invocaciones al mismo método. Sin embargo, con un único bloque `try{}` el código es más legible.

7.4 Gestión de Excepciones: Lanzar

Una vez se invoque desde *método_A* el *método_B* comenzará la ejecución de su código, que tendrá 3 partes:

- **Detección de la excepción:** se especifican las condiciones en las cuales se genera la excepción
- **Generación de la excepción:** si se cumplen esas condiciones, se crea un objeto del tipo de excepción correspondiente. Es habitual pasar información del contexto en el que ocurrió la excepción como argumentos de su constructor, como un mensaje y/o referencias a objetos que se podrán usar en el tratamiento de la excepción
- **Lanzamiento de la excepción:** el objeto creado se transfiere al trozo de código en el que se trata la excepción usando `throw(excepcion)`. Al ejecutar `throw()` se interrumpe la ejecución del *método_B*. El flujo del programa continuará en el código que lleva a cabo el tratamiento de la excepción.

7.5 Gestión de Excepciones: Tratar

El bloque `catch(<tipo> nombre){}` se encarga de **capturar el objeto excepción** lanzado por el *método*_B y contiene el código en el que se **trata la excepción**, el cual suele consistir en usar `getMessage()` para mostrar al usuario y/o grabar en un archivo el mensaje que se pasó como argumento al constructor de la excepción. El bloque `catch(){}` puede estar en el *método*_A o en cualquier otro método de la secuencia de invocaciones que llevó a la ocurrencia de la excepción.

Los bloques `catch(){}` están directamente asociados a los bloques `try{}`, con las siguientes restricciones:

- Un bloque `catch(){}` tiene que estar asociado a un único bloque `try{}`
- Un bloque `try{}` tiene que estar asociado, al menos, a un bloque `catch(){}`
- Un bloque `try()` podría estar asociado a tantos bloques `catch(){}` como excepciones distintas se puedan lanzar en él
- Un bloque `try()` podría estar asociado a menos bloques `catch(){}` como excepciones distintas se puedan lanzar en él

Si varias de estas excepciones tienen la misma clase base, entonces puede definirse un único bloque `catch(){}` para todas ellas con su clase base.

Si varias de estas excepciones tienen el mismo tratamiento, entonces se puede usar `multi-catch: catch(<excepcion1> | <excepcion2> | ...){... tratamiento ...}`. El `multi-catch` no añade ninguna semántica a la gestión de las excepciones, simplemente simplifica el código del bloque `catch{}`

La **herencia** y **polimorfismo** juegan un papel importante a la hora de decidir qué bloque `catch(){}` se ejecutará cuando el *método*_B lance una excepción:

- Si el argumento del bloque `catch(){}` es una clase superior a una clase *A*, después de él no podrá existir otro cuyo argumento sea la clase *A* o clases inferiores, pues la excepción ya se habrá capturado en el primero. Añadirlo provocaría un **error de compilación**.

En el tratamiento de las excepciones, además de los bloques `catch(){}` también existe un bloque `finally{}` que se ejecuta siempre, independientemente de la excepción que se haya capturado o incluso si no se ha capturado ninguna.

- Un bloque `finally{}` tiene que estar asociado a un único bloque `try{}`
- Un bloque `try{}` puede estar asociado a un único bloque `finally`
- Un bloque `finally{}` siempre va después de todos los bloques `catch(){}` asociados al `try{}`

El bloque `finally{}` suele usarse para liberar recursos (por ejemplo, cerrar archivos, finalizar una conexión con una base de datos, etc.)

7.6 Gestión de Excepciones: Proceso Completo

El *método*_A no tiene por qué tener un bloque `try{}-catch(){}` en el que gestione las excepciones del *método*_B, puede **delegar** su captura y tratamiento a otro método que lo invoque indicando que él mismo provoca las mismas excepciones que el *método*_B

Protips para el Final

Conceptos Clave de Teoría

Encapsulación

Wrappers-> Unboxing y Autoboxing

Referencias

Strings y String Pool

Método equals

Reservar Memoria Para Arrays

Que TD es iterable y cual no

Herencia

Clase Abstracta

Polimorfismo-> downcasting upcasting

Herencia Multiple en Interfaces

Excepciones extienden de Exception

HashMaps

Recorrerlo usando las claves:

```

public class Main{
    private HashMap<Integer, String> mapa;

    public void main(String[] args){
        mapa= new HashMap<>();

        Set<Integer> claves = mapa.keySet();
        for(Integer int: claves){
            String str = mapa.get(int);
            System.out.println(str);
        }
    }
}

```

Recorrerlo usando los valores

```

public class Main{
    private HashMap<Integer, String> mapa;

    public void main(String[] args){
        mapa= new HashMap<>();

        Collection<String> valores = mapa.values();
        for(String str: valores){
            System.out.println(str);
        }
    }
}

```

Recorrerlo usando la dupla de valores (mapa de entrada)

```

public class Main{
    private HashMap<Integer, String> mapa;

    public void main(String[] args){
        mapa= new HashMap<>();

        Set<Map.Entry<Integer,String>> valores = mapa.entrySet();
        for(Map.Entry dupla: valores){
            String str = dupla.getValue(); //dupla.getKey();
            System.out.println(str);
        }
    }
}

```

```
    }  
}
```

Recorrerlo Usando Iterator

```
public class Main{  
    private HashMap<Integer, String> mapa;  
  
    public void main(String[] args){  
        mapa= new HashMap<>();  
  
        Set<Integer> valores = mapa.keySet();  
        Iterator<Integer> it = valores.iterator();  
  
        while(it.hasNext()){  
            String str = it.next();  
            System.out.println(str);  
        }  
    }  
}
```

Jerarquías

```
public abstract class Vehiculo {  
    private String nombre;  
    private int valor;  
  
    // Constructor sin parámetros  
    public Vehiculo() {  
    }  
  
    // Constructor con parámetros  
    public Vehiculo(String nombre, int valor) {  
        this.nombre = nombre;  
        this.valor = valor;  
    }  
  
    // Métodos getter y setter  
    public String getNombre() {  
        return nombre;  
    }  
}
```



```

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public int getValor() {
        return valor;
    }

    public void setValor(int valor) {
        this.valor = valor;
    }

    // Método abstracto que será implementado por las clases hijas
    public abstract int calcularValor();

    // Método común para imprimir detalles del vehículo
    public void imprimirDetalles() {
        System.out.println("Nombre: " + nombre + ", Valor base: " + valor);
    }
}

```

```

public class Coche extends Vehiculo {
    private boolean desembrague;

    // Constructor sin parámetros
    public Coche() {
        super();
    }

    // Constructor con parámetros
    public Coche(String nombre, int valor, boolean desembrague) {
        super(nombre, valor);
        this.desembrague = desembrague;
    }

    // Getter y setter
    public boolean isDesembrague() {
        return desembrague;
    }

    public void setDesembrague(boolean desembrague) {
        this.desembrague = desembrague;
    }
}

```

```

    }

    // Implementación del método abstracto
    @Override
    public int calcularValor() {
        return getValor() + (desembrague ? 1000 : 0); // Valor adicional si tiene
desembrague
    }

    // Sobrescritura del método imprimirDetalles
    @Override
    public void imprimirDetalles() {
        super.imprimirDetalles();
        System.out.println("Desembrague: " + (desembrague ? "Sí" : "No"));
    }
}

```

```

public class Moto extends Vehiculo {
    private boolean anchoDeManubrio;

    // Constructor sin parámetros
    public Moto() {
        super();
    }

    // Constructor con parámetros
    public Moto(String nombre, int valor, boolean anchoDeManubrio) {
        super(nombre, valor);
        this.anchoDeManubrio = anchoDeManubrio;
    }

    // Getter y setter
    public boolean isAnchoDeManubrio() {
        return anchoDeManubrio;
    }

    public void setAnchoDeManubrio(boolean anchoDeManubrio) {
        this.anchoDeManubrio = anchoDeManubrio;
    }

    // Implementación del método abstracto
    @Override
    public int calcularValor() {

```

```

        return getValor() + (anchoDeManubrio ? 500 : 0); // Valor adicional si tiene
manubrio ancho
    }

    // Sobrescritura del método imprimirDetalles
    @Override
    public void imprimirDetalles() {
        super.imprimirDetalles();
        System.out.println("Manubrio ancho: " + (anchoDeManubrio ? "Sí" : "No"));
    }

    public String superreventao(){
        System.out.println("Primero que nada que le follen al Estado");
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        Vehiculo V1 = new Coche("Toyota", 20000, true); //Upcasting
        Vehiculo V2 = new Moto("Yamaha", 15000, false);

        V1.imprimirDetalles();
        System.out.println("Valor total del coche: " + coche.calcularValor());

        V2.superreventao(); //dará warning y no compila

        Moto moto = (Moto) V2; //downcasting
        moto.superreventao(); //lo ejecutará sin problema
    }
}

```

Excepciones

```

// Clase de excepción personalizada
public class VehiculoException extends Exception {
    public VehiculoException(String mensaje) {
        super(mensaje);
    }
    // Método adicional para ayudar a resolver el problema

    public String sugerirSolucion() {

```

```

        return "Revise los parámetros de entrada y asegúrese de que sean
válidos.";
    }
}

```

En vehiculo:

```

public Vehiculo(String nombre, int valor) throws VehiculoException {
    if (valor < 0) {
        throw new VehiculoException("El valor del vehículo no puede ser
negativo.");
    }
    this.nombre = nombre;
    this.valor = valor;
}

public void setValor(int valor) throws VehiculoException {
    if (valor < 0) {
        throw new VehiculoException("El valor del vehículo no puede ser
negativo.");
    }
    this.valor = valor;
}

```

```

// Clase principal para demostrar manejo de excepciones
public class Main { public static void main(String[] args) {
    try {
        // Intentamos crear un vehículo con un valor negativo
        Vehiculo coche = new Coche("Toyota", -20000, true);
    } catch (VehiculoException e) {
        // Capturamos la excepción y mostramos un mensaje
        System.out.println("Error al crear el vehículo: " +
e.getMessage());
        System.out.println("Sugerencia: " + e.sugerirSolucion());
    } try {
        // Crear un vehículo correctamente
        Vehiculo moto = new Coche("Honda", 15000, false); // Intentar
asignar un valor negativo
        moto.setValor(-5000);
    } catch (VehiculoException e) { // Capturar y manejar la excepción
        System.out.println("Error al actualizar el valor del vehículo: " +
e.getMessage());
    }
}
}

```

```

        System.out.println("Sugerencia: " + e.sugerirSolucion()); //
Opcionalmente, podemos volver a lanzar la excepción si es necesario
        throw new RuntimeException("Excepción no manejada", e);
    }

    System.out.println("Programa finalizado.");
}
}

```

Interfaces

```

// Definición de la interfaz
public interface VehiculoInterface {
    // Método abstracto
    void mover();

    // Método por defecto
    default void encender() {
        System.out.println("El vehículo está encendido.");
    }

    // Método estático
    static void mostrarTipoVehiculo() {
        System.out.println("Todos los vehículos tienen ruedas.");
    }
}

// Clase Coche que implementa la interfaz
public class Coche implements VehiculoInterface {
    private String modelo;

    public Coche(String modelo) {
        this.modelo = modelo;
    }

    @Override
    public void mover() {
        System.out.println("El coche " + modelo + " se está moviendo.");
    }
}

// Clase Moto que implementa la interfaz
public class Moto implements VehiculoInterface {

```

```

private String marca;

public Moto(String marca) {
    this.marca = marca;
}

@Override
public void mover() {
    System.out.println("La moto " + marca + " se está moviendo.");
}

// Sobrescribiendo el método por defecto
@Override
public void encender() {
    System.out.println("La moto está encendida de manera personalizada.");
}
}

// Clase principal para demostrar el uso de la interfaz
public class Main {
    public static void main(String[] args) {
        // Uso del polimorfismo: referencias de tipo VehiculoInterface
        VehiculoInterface coche = new Coche("Toyota");
        VehiculoInterface moto = new Moto("Yamaha");

        // Métodos abstractos
        coche.mover();
        moto.mover();

        // Métodos por defecto
        coche.encender();
        moto.encender();

        // Método estático de la interfaz
        VehiculoInterface.mostrarTipoVehiculo();

        // Uso adicional para demostrar independencia
        realizarAccion(coche);
        realizarAccion(moto);
    }

    // Método que usa polimorfismo para trabajar con VehiculoInterface
    public static void realizarAccion(VehiculoInterface vehiculo) {

```

```
        System.out.println("Acción genérica para cualquier vehículo:");  
        vehiculo.mover();  
    }  
}
```